Connect Claude Code to Tools via MCP

Documentation Index

Fetch the complete documentation index at:

```
https://code.claude.com/docs/llms.txt
```

Use this file to discover all available pages before exploring further.

Claude Code can connect to hundreds of external tools and data sources through the Model Context Protocol (MCP), an open-source standard for AI-tool integrations.

MCP servers give Claude Code access to your tools, databases, and APIs.

Connect a server when you find yourself copying data into chat from another tool, such as an issue tracker or monitoring dashboard. Once connected, Claude can read and act on that system directly instead of working from what you paste.

For your first server, start with the MCP quickstart for a step-by-step walkthrough. This page is the full reference.

What You Can Do with MCP

With MCP servers connected, you can ask Claude Code to:

- **Implement features from issue trackers**

  Add the feature described in JIRA issue ENG-4521 and create a PR on GitHub.

- **Analyze monitoring data**

  Check Sentry and Statsig to check the usage of the feature described in ENG-4521.

- **Query databases**

  Find emails of 10 random users who used feature ENG-4521, based on our PostgreSQL database.

- **Integrate designs**

Update our standard email template based on the new Figma designs that were posted in Slack.

- **Automate workflows**

  Create Gmail drafts inviting these 10 users to a feedback session about the new feature.

- **React to external events**

An MCP server can also act as a channel that pushes messages into your session, allowing Claude to react to Telegram messages, Discord chats, or webhook events while you're away.

Find and Build MCP Servers

Browse reviewed connectors in the Anthropic Directory.

Directory connectors use the same MCP infrastructure as Claude Code, so you can add any remote server listed there with:

```
claude mcp add
```

Warning

Verify that you trust each server before connecting it. Servers that fetch external content can expose you to prompt injection risk.

To build your own server, see:

- MCP server guide for protocol fundamentals
- Claude connector building docs for authentication, testing, and Directory submission

You can also have Claude scaffold a server using the official `mcp-server-dev` plugin.

Install the Plugin

In a Claude Code session, run:

```
/plugin install mcp-server-dev@claude-plugins-official
```

If Claude Code reports that the marketplace is not found, run:

```
/plugin marketplace add anthropics/claude-plugins-official
```

Then retry the installation.

Once installed, activate it in the current session:

```
/reload-plugins
```

Run the Build Skill

```
/mcp-server-dev:build-mcp-server
```

Claude asks about your use case and scaffolds either:

- a remote HTTP server, or
- a local stdio server

Installing MCP Servers

MCP servers can be configured in several ways depending on your needs.

Option 1: Add a Remote HTTP Server

HTTP servers are the recommended option for connecting to remote MCP servers. This is the most widely supported transport for cloud-based services.

```
# Basic syntax
claude mcp add --transport http <name> <url>

# Real example: Connect to Notion
claude mcp add --transport http notion https://mcp.notion.com/mcp

# Example with Bearer token
claude mcp add --transport http secure-api https://api.example.com/mcp
  --header "Authorization: Bearer your-token"
```

When configuring MCP servers via JSON in:

- `.mcp.json`

- `~/.claude.json`
- `claude mcp add-json`

the `type` field accepts `streamable-http` as an alias for `http`.

The MCP specification uses the name `streamable-http`, so configurations copied from server documentation work without modification.

A JSON entry that has a `url` but no `type` is a configuration error because Claude Code reads an entry with no `type` as a stdio server.

Claude Code skips that server and reports:

```
MCP server "<name>" has a "url" but no "type"; add "type": "http" (or "sse" /
"ws") to this entry
```

Older versions reported:

```
command: expected string, received undefined
```

Option 2: Add a Remote SSE Server

> **Warning**
>
> The SSE transport is deprecated. Use HTTP servers instead where available.

```
# Basic syntax
claude mcp add --transport sse <name> <url>

# Real example: Connect to Asana
claude mcp add --transport sse asana https://mcp.asana.com/sse

# Example with authentication header
claude mcp add --transport sse private-api https://api.company.com/sse
  --header "X-API-Key: your-key-here"
```

Option 3: Add a Local stdio Server

Stdio servers run as local processes on your machine. They're ideal for tools that need:

- direct system access
- local filesystem access
- custom scripts

Claude Code sets:

```
CLAUDE_PROJECT_DIR
```

in the spawned server's environment.

This points to the project root, allowing your server to resolve project-relative paths without depending on the current working directory.

Examples:

```
process.env.CLAUDE_PROJECT_DIR
```

```
os.environ["CLAUDE_PROJECT_DIR"]
```

`CLAUDE_PROJECT_DIR` is the stable project root. It does not change when you add or remove working directories during a session.

A server that limits its filesystem access to a set of allowed directories should implement the MCP:

```
roots/list
```

request instead.

Claude Code answers `roots/list` with:

- the session's launch directory
- every additional working directory granted through:
- `--add-dir`
- `/add-dir`
- the `additionalDirectories` setting

Claude Code sends:

```
notifications/roots/list_changed
```

when that set changes.

`CLAUDE_PROJECT_DIR` is set in the server's environment, not Claude Code's own environment.

Therefore, referencing it through `${VAR}` expansion in a project- or user-scoped `.mcp.json` `command` or `args` should use a fallback:

```
${CLAUDE_PROJECT_DIR:-.}
```

Plugin-provided MCP configurations can substitute:

```
${CLAUDE_PROJECT_DIR}
```

directly without the fallback.

Basic Syntax

```
claude mcp add [options] <name> -- <command> [args...]
```

Example: Airtable Server

```
claude mcp add --env AIRTABLE_API_KEY=YOUR_KEY --transport stdio airtable
  -- npx -y airtable-mcp-server
```

Important: Separate Server Arguments with `--`

For stdio servers, the double dash:

```
--
```

separates Claude Code's options from the command and arguments that run the server.

Everything after `--` is passed to the server unchanged.

Examples:

```
claude mcp add --transport stdio myserver -- npx server
```

Runs:

```
npx server
```

Another example:

```
claude mcp add --env KEY=value --transport stdio myserver
  -- python server.py --port 8080
```

Runs:

```
python server.py --port 8080
```

with:

```
KEY=value
```

in the server environment.

Without `--`, Claude Code may try to interpret server flags such as:

```
--port
```

as Claude Code options.

`--env` accepts multiple `KEY=value` pairs.

If the server name comes directly after `--env`, the CLI may interpret the name as another environment-variable pair and reject it. Place at least one other option between `--env` and the server name.

Option 4: Add a Remote WebSocket Server

WebSocket servers maintain a persistent bidirectional connection.

This is useful for remote MCP servers that push events to Claude without being prompted.

Use HTTP instead when the server only responds to requests because:

- HTTP supports OAuth
- HTTP supports the `claude mcp add --transport` flag
- WebSocket supports neither

Configure WebSocket servers through:

- `.mcp.json`
- `claude mcp add-json`

Example:

```
claude mcp add-json events-server
  '{"type":"ws","url":"wss://mcp.example.com/socket","headers":
{"Authorization":"Bearer YOUR_TOKEN"}}'
```

The `type: "ws"` entry accepts the same fields as HTTP:

- `url`
- `headers`
- `headersHelper`
- `timeout`
- `alwaysLoad`

Authentication is header-only.

Use either:

- a static token in `headers`, or
- a dynamically generated token through `headersHelper`

The `claude mcp add --transport` flag does not accept `ws`.

Managing Your Servers

Once configured, manage MCP servers with:

```
# List all configured servers
claude mcp list

# Get details for a specific server
claude mcp get github
```

```
# Remove a server
claude mcp remove github
```

Inside Claude Code:

```
/mcp
```

Project-Scoped Server Approval

Project-scoped servers from `.mcp.json` that are awaiting approval appear as:

```
" Pending approval
```

Run Claude interactively:

```
claude
```

to review and approve them.

The following command also shows approval state:

```
claude mcp get <name>
```

Possible statuses include:

```
" Pending approval
```

and:

```
✗ Rejected
```

Claude Code reads `.mcp.json` approvals only from settings files that are not checked into the repository until you trust the workspace by:

1. running `claude` in the workspace, and
2. accepting the workspace trust dialog

A cloned repository cannot approve its own MCP servers.

For example, these settings committed to:

```
.claude/settings.json
```

are ignored in an untrusted folder:

```
enableAllProjectMcpServers
enabledMcpjsonServers
```

The server remains:

```
" Pending approval
```

until the workspace is trusted.

Approvals from these sources still apply in an untrusted folder:

- `~/.claude/settings.json`
- managed settings
- settings supplied through `--settings`

Approvals in:

```
.claude/settings.local.json
```

also apply, but only after you accept a trust dialog for that folder or one of its parent directories.

An exception exists when the folder is your own Claude configuration home:

- your home directory, or
- a directory whose `.claude` directory is configured through `CLAUDE_CONFIG_DIR`

A:

```
disabledMcpjsonServers
```

entry in any settings file still rejects the server.

The `/mcp` panel:

- shows the tool count next to each connected server

- flags servers that advertise tool support but expose no tools

---

Waiting for MCP Servers

If your request needs tools from a server that is still connecting, Claude waits for that server before continuing.

With tool search enabled, the wait occurs inside:

```
ToolSearch
```

In configurations without tool search, Claude uses:

```
WaitForMcpServers
```

Examples of environments where tool search may be unavailable include:

- Google Cloud's Agent Platform
- a custom `ANTHROPIC_BASE_URL`
- `ENABLE_TOOL_SEARCH=false`

Reserved Server Names

Some server names are reserved for Claude Code's built-in servers:

```
workspace
claude-in-chrome
computer-use
Claude Preview
Claude Browser
```

If your configuration uses a reserved name, Claude Code skips the server and warns you to rename it.

`claude mcp add` rejects reserved names directly.

`Claude Preview` and `Claude Browser` both refer to the built-in server used by the Claude Code desktop application's preview pane.

Dynamic Tool Updates

Claude Code supports MCP `list_changed` notifications.

This allows MCP servers to dynamically update their available:

- tools
- prompts
- resources

without requiring you to disconnect and reconnect.

When an MCP server sends a `list_changed` notification, Claude Code automatically refreshes the available capabilities.

Automatic Reconnection

If an HTTP or SSE server disconnects during a session, Claude Code automatically reconnects using exponential backoff.

It attempts up to five reconnections:

```
1 second
2 seconds
4 seconds
8 seconds
16 seconds
```

The server appears as pending in `/mcp` while reconnection is in progress.

After five failed attempts, the server is marked as failed and can be retried manually from `/mcp`.

Stdio servers are local processes and are not automatically reconnected.

The same backoff logic applies when an HTTP or SSE server fails its initial connection at startup.

Claude Code retries transient failures such as:

- HTTP 5xx responses
- connection refused
- timeout

Authentication and not-found errors are not retried because they usually require configuration changes.

When a configured server fails to connect, Claude Code can expose:

- which server failed
- the connection error

This information may also appear in `ToolSearch` results when no matching tool is found.

Capability-discovery requests such as:

```
tools/list
prompts/list
resources/list
```

also retry transient network and server errors.

Authentication errors, 4xx errors, and request timeouts are not retried.

Push Messages with Channels

An MCP server can push messages directly into your session.

This allows Claude to react to external events such as:

- CI results
- monitoring alerts
- chat messages
- webhook events

To enable this, the server declares:

```
claude/channel
```

and you opt in using:

```
--channels
```

See:

- Channels
- Channels reference

Useful MCP Configuration Tips

Choose a Scope

Use:

```
-s
```

or:

```
--scope
```

Available scopes:

- `local`
- `project`
- `user`

Local

Available only to you in the current project.

Older versions called this:

```
project
```

Project

Shared with everyone in the project through:

```
.mcp.json
```

User

Available to you across all projects.

Older versions called this:

```
global
```

Set Environment Variables

Use:

```
-e
```

or:

```
--env
```

Example:

```
-e KEY=value
```

Short Transport and Header Flags

```
-t
```

is the short form of:

```
--transport
```

```
-H
```

is the short form of:

```
--header
```

Configure MCP Startup Timeout

Use:

```
MCP_TIMEOUT=10000 claude
```

This example sets a 10-second startup timeout.

Configure Per-Server Tool Timeout

Add a `timeout` field in milliseconds:

```
{
  "timeout": 600000
}
```

This example sets a 10-minute timeout.

The per-server value overrides:

```
MCP_TOOL_TIMEOUT
```

for that server.

The per-server timeout is a hard wall-clock limit per tool call.

Progress notifications do not extend it.

Values below:

```
1000
```

milliseconds are ignored and fall through to:

```
MCP_TOOL_TIMEOUT
```

or its default.

Per-Request Timeout

For:

- HTTP
- SSE
- claude.ai connectors

there is also a per-request timer covering the request until the server's first response byte.

The default is:

```
60 seconds
```

Setting either:

```
timeout
```

or:

```
MCP_TOOL_TIMEOUT
```

to 60 seconds or more raises the per-request timer to that value.

A lower value does not shorten the default 60-second timer.

Stdio and WebSocket servers do not have this separate per-request timer.

Idle Timeout

An MCP tool call that sends:

- no response
- no progress notification

for the idle window is aborted instead of waiting indefinitely for the wall-clock timeout.

The default idle windows are:

Server Type	Default Idle Timeout
HTTP	5 minutes

Server Type	Default Idle Timeout
SSE	5 minutes
WebSocket	5 minutes
claude.ai connector	5 minutes
stdio	30 minutes

Change the idle window with:

```
CLAUDE_CODE_MCP_TOOL_IDLE_TIMEOUT
```

The value is expressed in milliseconds.

Disable the idle check with:

```
CLAUDE_CODE_MCP_TOOL_IDLE_TIMEOUT=0
```

A per-server `timeout` of at least 1000 milliseconds also acts as a floor for the idle timeout.

Increase MCP Output Limits

Claude Code warns when MCP tool output exceeds:

```
10,000 tokens
```

Increase the limit with:

```
MAX_MCP_OUTPUT_TOKENS=50000 claude
```

Plugin-Provided MCP Servers

Plugins can bundle MCP servers and automatically provide tools and integrations when enabled.

Plugin MCP servers work like manually configured MCP servers.

How Plugin MCP Servers Work

Plugins can define MCP servers:

- in `.mcp.json` at the plugin root, or
- inline in `plugin.json`

When the plugin is enabled:

- its MCP servers start automatically
- its MCP tools appear alongside manually configured tools

Plugin MCP servers are managed through plugin installation rather than `/mcp` configuration commands.

Example Plugin MCP Configuration

`.mcp.json`

```
{
  "mcpServers": {
    "database-tools": {
      "command": "${CLAUDE_PLUGIN_ROOT}/servers/db-server",
      "args": ["--config", "${CLAUDE_PLUGIN_ROOT}/config.json"],
      "env": {
        "DB_URL": "${DB_URL}"
      }
    }
  }
}
```

Inline in `plugin.json`

```
{
  "name": "my-plugin",
  "mcpServers": {
    "plugin-api": {
      "command": "${CLAUDE_PLUGIN_ROOT}/servers/api-server",
      "args": ["--port", "8080"]
    }
  }
}
```

Plugin MCP Features

Automatic Lifecycle

At session startup, servers for enabled plugins connect automatically.

After enabling or disabling a plugin during a session, run:

```
/reload-plugins
```

to connect or disconnect its MCP servers.

Environment Variables

Use:

```
${CLAUDE_PLUGIN_ROOT}
```

for bundled plugin files.

Use:

```
${CLAUDE_PLUGIN_DATA}
```

for persistent state that survives plugin updates.

Use:

```
${CLAUDE_PROJECT_DIR}
```

for the stable project root.

User Environment Access

Plugin MCP servers have access to the same user environment variables as manually configured servers.

Multiple Transport Types

Plugin MCP servers can support:

- stdio
- SSE

- HTTP
- WebSocket

Transport support may vary by server.

Viewing Plugin MCP Servers

Inside Claude Code:

```
/mcp
```

Plugin servers appear with indicators showing that they came from plugins.

Plugin MCP Tool Names

Tools from a plugin-bundled MCP server include both:

- the plugin name
- the server key

The full format is:

```
mcp__plugin_<plugin-name>_<server-name>__<tool-name>
```

Any character outside:

```
A-Z
a-z
0-9
_
-
```

is replaced with:

```
_
```

Example:

- Plugin: `my-plugin`
- Server: `database-tools`

- Tool: `query`

Callable name:

```
mcp__plugin_my-plugin_database-tools__query
```

Use this full name when referencing the tool in:

- permission rules
- a skill's `allowed-tools`
- a subagent's `tools` field
- a hook matcher

A matcher written against only the bare server key:

```
mcp__database-tools__.*
```

will not match a plugin-bundled server.

The server itself registers under a scoped name:

```
plugin:<plugin-name>:<server-name>
```

Example:

```
plugin:my-plugin:database-tools
```

Use this name wherever a configured server name is expected.

Benefits of Plugin MCP Servers

- **Bundled distribution** — tools and servers are packaged together
- **Automatic setup** — no manual MCP configuration is required
- **Team consistency** — everyone receives the same tools when the plugin is installed

See the plugin components reference for more information.

MCP Installation Scopes

MCP servers can be configured at three scopes.

Scope	Loads In	Shared with Team	Stored In
Local	Current project only	No	`~/.claude.json`
Project	Current project only	Yes, via version control	`.mcp.json`
User	All projects	No	`~/.claude.json`

Administrators can also deploy enterprise-level servers through managed configuration.

Local Scope

Local scope is the default.

A local-scoped server:

- loads only in the project where you added it
- stays private to you
- is stored in `~/.claude.json` under that project's path

Use local scope for:

- personal development servers
- experiments
- credentials you do not want committed to version control

> **Note**
>
> MCP local scope differs from Claude Code's general local settings.
>
> MCP local-scoped servers are stored in:
>
> ```
> ~/.claude.json
> ```
>
> General local settings are stored in:
>
> ```
> .claude/settings.local.json
> ```

Add a Local-Scoped Server

```
# Local is the default
claude mcp add --transport http stripe https://mcp.stripe.com
```

```
# Explicitly specify local scope
claude mcp add --transport http stripe --scope local https://mcp.stripe.com
```

Example resulting configuration:

```
{
  "projects": {
    "/path/to/your/project": {
      "mcpServers": {
        "stripe": {
          "type": "http",
          "url": "https://mcp.stripe.com"
        }
      }
    }
  }
}
```

Project Scope

Project-scoped servers support team collaboration.

Their configuration is stored in:

```
.mcp.json
```

at the project root.

This file can be checked into version control.

Example:

```
claude mcp add --transport http paypal --scope project
  https://mcp.paypal.com/mcp
```

Result:

```
{
  "mcpServers": {
    "shared-server": {
```

```
      "command": "/path/to/server",
      "args": [],
      "env": {}
    }
  }
}
```

For security reasons, Claude Code asks for approval before using project-scoped servers from `.mcp.json`.

Reset approval choices with:

```
claude mcp reset-project-choices
```

User Scope

User-scoped servers are:

- stored in `~/.claude.json`
- available across all your projects
- private to your user account

Use user scope for:

- personal utility servers
- development tools
- frequently used services

Example:

```
claude mcp add --transport http hubspot --scope user
  https://mcp.hubspot.com/anthropic
```

Scope Hierarchy and Precedence

When the same server is defined in more than one place, Claude Code connects to it once using the highest-precedence definition.

The server entry is taken as a whole.

Fields are not merged across scopes.

Precedence:

1. Local scope
2. Project scope
3. User scope
4. Plugin-provided servers
5. claude.ai connectors

The three regular scopes match duplicates by server name.

Plugins and connectors match duplicates by endpoint.

Therefore, a plugin or connector that points to the same URL or command as a higher-precedence server is treated as a duplicate.

Environment Variable Expansion in `.mcp.json`

Claude Code supports environment-variable expansion in `.mcp.json`.

This allows teams to share configurations while keeping:

- machine-specific paths
- API keys
- sensitive values

outside the repository.

Supported Syntax

```
${VAR}
```

Expands to the value of `VAR`.

```
${VAR:-default}
```

Expands to `VAR` when set, otherwise uses `default`.

Supported Locations

Environment variables can be expanded in:

- `command`

- `args`
- `env`
- `url`
- `headers`

Example:

```
{
  "mcpServers": {
    "api-server": {
      "type": "http",
      "url": "${API_BASE_URL:-https://api.example.com}/mcp",
      "headers": {
        "Authorization": "Bearer ${API_KEY}"
      }
    }
  }
}
```

If a referenced variable is not set and has no default value, Claude Code leaves the literal:

```
${VAR}
```

in the configuration and reports a missing-variable warning.

The configuration still loads.

Set the variable or provide a fallback to make the server start with the intended value.

Practical Examples

Example: Monitor Errors with Sentry

Add the server:

```
claude mcp add --transport http sentry https://mcp.sentry.dev/mcp
```

Authenticate:

```
/mcp
```

Then ask:

```
What are the most common errors in the last 24 hours?
```

```
Show me the stack trace for error ID abc123
```

```
Which deployment introduced these new errors?
```

Example: Connect to GitHub for Code Reviews

GitHub's remote MCP server authenticates using a GitHub personal access token passed as a header.

Generate a fine-grained token with access to the repositories you want Claude to use.

Then add the server:

```
claude mcp add --transport http github https://api.githubcopilot.com/mcp/
  --header "Authorization: Bearer YOUR_GITHUB_PAT"
```

Example prompts:

```
Review PR #456 and suggest improvements
```

```
Create a new issue for the bug we just found
```

```
Show me all open PRs assigned to me
```

Example: Query PostgreSQL

```
claude mcp add --transport stdio db -- npx -y @bytebase/dbhub
  --dsn "postgresql://readonly:pass@prod.db.com:5432/analytics"
```

Then ask:

```
What's our total revenue this month?
```

```
Show me the schema for the orders table
```

```
Find customers who haven't made a purchase in 90 days
```

Authenticate with Remote MCP Servers

Many cloud-based MCP servers require authentication.

Claude Code supports OAuth 2.0.

A remote server is marked as needing authentication when it responds with:

```
401 Unauthorized
```

or:

```
403 Forbidden
```

For a server you have not signed into, either response flags it in `/mcp`.

If a server you previously authenticated with later returns:

```
401 Unauthorized
```

Claude Code:

1. refreshes the stored token
2. reconnects
3. retries the request once

The server is flagged in `/mcp` only if the retry also fails.

If the server rejects the stored refresh token, Claude Code directs you to:

```
/mcp
```

where the server menu provides:

```
Re-authenticate
```

A custom server that returns a `WWW-Authenticate` header pointing to its authorization server receives the same automatic discovery behavior.

Claude Code can also display a startup notice when configured servers require authentication.

Authentication in Non-Interactive Mode

In:

```
claude -p
```

or Agent SDK runs, there is no `/mcp` panel.

When tool search is enabled, Claude Code can tell Claude that a server's tools are unavailable until authorization is completed.

Authenticate from an interactive session using:

```
/mcp
```

or:

```
claude mcp login <name>
```

If you configured:

```
headers.Authorization
```

and the server rejects that header, Claude Code reports the connection as failed rather than falling back to OAuth.

Either:

- correct the token, or
- remove the static authorization header and use OAuth

Standard OAuth Flow

Add the Server

```
claude mcp add --transport http sentry https://mcp.sentry.dev/mcp
```

Authenticate Inside Claude Code

```
/mcp
```

Follow the browser login flow.

Authentication Tips

- Tokens are stored securely and refreshed automatically.
- Use **Clear authentication** in `/mcp` to revoke access.
- If the browser does not open automatically, copy the URL and open it manually.
- If the browser redirect fails, paste the full callback URL from the browser address bar into Claude Code.
- OAuth authentication works with HTTP servers.

Authenticate from the Command Line

Run:

```
claude mcp login sentry
```

Clear stored credentials with:

```
claude mcp logout sentry
```

During SSH sessions or on Linux systems without a browser, Claude Code can print the authorization URL instead.

Open the URL on your local machine, complete authentication, and paste the full redirect URL back into the terminal.

For SSH, use an interactive terminal:

```
ssh -t
```

Force URL-based authentication with:

```
claude mcp login sentry --no-browser
```

Use a Fixed OAuth Callback Port

Some MCP servers require a pre-registered redirect URI.

By default, Claude Code chooses a random available port.

Use:

```
--callback-port
```

to select a fixed port.

The redirect URI is:

```
http://localhost:PORT/callback
```

Example:

```
claude mcp add --transport http
  --callback-port 8080
  my-server https://mcp.example.com/mcp
```

Use Pre-Configured OAuth Credentials

Some MCP servers do not support Dynamic Client Registration.

You may see an error such as:

```
Incompatible auth server: does not support dynamic client registration
```

In that case:

1. register an OAuth application through the server's developer portal
2. obtain the client ID and, when applicable, the client secret
3. register the callback URI
4. add the server with those credentials

Using `claude mcp add`

```
claude mcp add --transport http
  --client-id your-client-id
  --client-secret
  --callback-port 8080
  my-server https://mcp.example.com/mcp
```

The `--client-secret` flag prompts for the secret using masked input.

Using `claude mcp add-json`

```
claude mcp add-json my-server
  '{"type":"http","url":"https://mcp.example.com/mcp","oauth":{"clientId":"your-
client-id","callbackPort":8080}}'
  --client-secret
```

Callback Port Only

```
claude mcp add-json my-server
  '{"type":"http","url":"https://mcp.example.com/mcp","oauth":{"callbackPort":
8080}}'
```

CI or Environment Variable

```
MCP_CLIENT_SECRET=your-secret claude mcp add --transport http
  --client-id your-client-id
  --client-secret
  --callback-port 8080
  my-server https://mcp.example.com/mcp
```

Then authenticate in Claude Code:

```
/mcp
```

OAuth Credential Tips

- The client secret is stored securely rather than in your MCP configuration.
- Public OAuth clients can use only `--client-id`.
- `--callback-port` can be used with or without `--client-id`.
- These flags apply only to HTTP and SSE transports.
- Verify configuration with:

```
claude mcp get <name>
```

Override OAuth Metadata Discovery

Use:

```
authServerMetadataUrl
```

to point Claude Code at a specific OAuth authorization-server metadata URL.

This can be useful when:

- standard discovery endpoints fail
- you need to route discovery through an internal proxy

Example:

```
{
  "mcpServers": {
```

```
    "my-server": {
      "type": "http",
      "url": "https://mcp.example.com/mcp",
      "oauth": {
        "authServerMetadataUrl": "https://auth.example.com/.well-known/openid-
configuration"
      }
    }
  }
}
```

The URL must use:

```
https://
```

The metadata URL's:

```
scopes_supported
```

overrides the scopes advertised by the upstream server.

Restrict OAuth Scopes

Set:

```
oauth.scopes
```

to explicitly control which scopes Claude Code requests.

The value is a single space-separated string.

Example:

```
{
  "mcpServers": {
    "slack": {
      "type": "http",
      "url": "https://mcp.slack.com/mcp",
      "oauth": {
        "scopes": "channels:read chat:write search:read"
```

```
      }
    }
  }
}
```

`oauth.scopes` takes precedence over:

- `authServerMetadataUrl`
- scopes discovered from `/.well-known` endpoints

Leave it unset to allow the MCP server to determine the requested scope set.

If the authorization server advertises:

```
offline_access
```

Claude Code appends it so the access token can be refreshed without requiring a new browser sign-in.

If the server later returns:

```
403 insufficient_scope
```

Claude Code re-authenticates using the same pinned scopes.

Expand `oauth.scopes` when a required tool needs additional permissions.

Use Dynamic Headers for Custom Authentication

For authentication schemes other than OAuth, such as:

- Kerberos
- short-lived tokens
- internal SSO

use:

```
headersHelper
```

Claude Code runs the specified command when connecting and merges the resulting headers into the connection.

Example:

```
{
  "mcpServers": {
    "internal-api": {
      "type": "http",
      "url": "https://mcp.internal.example.com",
      "headersHelper": "/opt/bin/get-mcp-auth-headers.sh"
    }
  }
}
```

Inline example:

```
{
  "mcpServers": {
    "internal-api": {
      "type": "http",
      "url": "https://mcp.internal.example.com",
      "headersHelper": "echo '{\"Authorization\": \"Bearer '\"$(get-token)
\"'\"}'"
    }
  }
}
```

Requirements

The helper command must:

- write a JSON object of string key-value pairs to stdout
- complete within 10 seconds

Dynamic headers override static `headers` with the same names.

The helper runs again:

- at session startup
- whenever the server reconnects

There is no built-in caching.

Your helper script is responsible for token reuse.

If a tool call returns:

```
401 Unauthorized
```

or:

```
403 Forbidden
```

Claude Code can:

1. rerun the helper
2. reconnect with fresh headers
3. retry the request once

The server is marked as needing authentication only if the retry also fails.

Environment Variables Available to `headersHelper`

Variable	Value
`CLAUDE_CODE_MCP_SERVER_NAME`	Name of the MCP server
`CLAUDE_CODE_MCP_SERVER_URL`	URL of the MCP server
`CLAUDE_PLUGIN_ROOT`	Plugin root, when provided by a plugin

These variables can be used to create a single helper script that supports multiple MCP servers.

For plugin-provided servers, the helper runs with its working directory set to the plugin root.

A plugin-provided `headersHelper` cannot reference:

```
${user_config.*}
```

because the command is executed through a shell.

Instead:

- place `${user_config.KEY}` in the server's `headers` field, or
- have the helper read the value from its environment or a configuration file

Note

`headersHelper` executes arbitrary shell commands.

At project or local scope, it runs only after you accept the workspace trust dialog.

Add MCP Servers from JSON Configuration

Add an HTTP Server

```
claude mcp add-json weather-api
  '{"type":"http","url":"https://api.weather.com/mcp","headers":
{"Authorization":"Bearer token"}}'
```

Add a stdio Server

```
claude mcp add-json local-weather
  '{"type":"stdio","command":"/path/to/weather-cli","args":["--api-
key","abc123"],"env":{"CACHE_DIR":"/tmp"}}'
```

Add an HTTP Server with OAuth Credentials

```
claude mcp add-json my-server
  '{"type":"http","url":"https://mcp.example.com/mcp","oauth":{"clientId":"your-
client-id","callbackPort":8080}}'
  --client-secret
```

Verify the Server

```
claude mcp get weather-api
```

Tips

- Make sure the JSON is correctly escaped for your shell.
- The JSON must conform to the MCP server configuration schema.
- Add `--scope user` to store the server in your user configuration.

Import MCP Servers from Claude Desktop

Run:

```
claude mcp add-from-claude-desktop
```

An interactive dialog allows you to select which servers to import.

Then verify:

```
claude mcp list
```

Server names added through `claude mcp` commands may contain only:

- letters
- numbers
- hyphens
- underscores

Claude Desktop does not impose the same restriction.

A Claude Desktop server name containing characters such as spaces cannot be imported.

The import reports rejected names and continues importing valid servers.

Tips

- This feature works only on macOS and Windows Subsystem for Linux.
- It reads Claude Desktop's standard configuration file.
- Use `--scope user` to add imported servers to your user configuration.
- Valid server names are preserved.
- Duplicate names receive a numerical suffix such as:

```
server_1
```

Use MCP Servers from claude.ai

When Claude Code is authenticated through a claude.ai account, MCP servers added in claude.ai can automatically become available in Claude Code.

Configure Connectors in claude.ai

Add servers at:

```
https://claude.ai/customize/connectors
```

On Team and Enterprise plans, only administrators may be able to add servers.

Authenticate the MCP Server

Complete any required authentication in claude.ai.

View Servers in Claude Code

Run:

```
/mcp
```

Servers from claude.ai appear with indicators showing their origin.

Connectors you have never signed into may be collapsed behind:

```
Show unused connectors
```

A connector you previously signed into remains visible even when reauthentication is required.

Authentication Method Matters

claude.ai connectors are fetched only when Claude Code's active authentication method is your claude.ai subscription.

They are not loaded when authentication is being provided through:

```
ANTHROPIC_API_KEY
ANTHROPIC_AUTH_TOKEN
apiKeyHelper
```

or third-party providers such as:

- Amazon Bedrock
- Google Cloud's Agent Platform

If `/mcp` does not show an expected connector:

1. run:

```
/status
```

1. check the active authentication method
2. unset conflicting environment variables or remove `apiKeyHelper`
3. run:

```
/login
```

1. select your claude.ai account

A server added directly in Claude Code takes precedence over a claude.ai connector pointing at the same URL.

When this happens, `/mcp` shows the connector as hidden and explains how to remove the duplicate.

Anthropic-Hosted Connectors

Some Anthropic-hosted connectors, such as:

- Microsoft 365
- Gmail
- Google Calendar

may not support local OAuth directly from Claude Code because the upstream identity provider only accepts the redirect URI registered by claude.ai.

In that case, connect them through:

```
Settings → Connectors
```

on claude.ai.

Once connected there, the connector appears automatically in Claude Code.

Disable claude.ai Connectors

Set:

```
{
  "disableClaudeAiConnectors": true
}
```

This setting uses **any-source-true** semantics.

A `true` value in any settings source takes precedence.

A checked-in project:

```
.claude/settings.json
```

can disable cloud connectors for the repository.

A project-level `false` cannot override a user- or policy-level `true`.

Servers explicitly provided with:

```
--mcp-config
```

are unaffected.

You can also disable connectors for a shell session:

```
ENABLE_CLAUDEAI_MCP_SERVERS=false claude
```

To block specific connectors instead of all connectors, use:

```
deniedMcpServers
```

by:

- server name
- URL pattern

You can also toggle a connector for the current project from:

```
/mcp
```

Note

These settings govern local Claude Code sessions.

Claude Code on the web provisions claude.ai connectors through the remote host as explicit `--mcp-config` entries, so `disableClaudeAiConnectors` does not apply there.

Connector URLs may also be rewritten through the session proxy, so URL-based deny rules targeting the original vendor URL may not match.

Manage cloud-session connector access through your claude.ai organization settings.

Use Claude Code as an MCP Server

Claude Code itself can run as an MCP server:

```
claude mcp serve
```

Example Claude Desktop configuration:

```
{
  "mcpServers": {
    "claude-code": {
      "type": "stdio",
      "command": "claude",
      "args": ["mcp", "serve"],
      "env": {}
    }
  }
}
```

Configure the Executable Path

The `command` field must point to the Claude Code executable.

Find its path with:

```
which claude
```

Then use the full path:

```
{
  "mcpServers": {
    "claude-code": {
      "type": "stdio",
      "command": "/full/path/to/claude",
      "args": ["mcp", "serve"],
      "env": {}
    }
  }
}
```

Without the correct executable path, you may see:

```
spawn claude ENOENT
```

Notes

- The server exposes Claude Code tools such as View, Edit, and LS.
- An MCP client can then ask Claude Code to read files, make edits, and perform related operations.
- The MCP client is responsible for implementing user confirmation for individual tool calls.

MCP Output Limits and Warnings

Claude Code helps manage large MCP outputs to avoid overwhelming the conversation context.

Default Thresholds

Setting	Default
Output warning threshold	10,000 tokens
Default maximum output	25,000 tokens

Increase the maximum:

```
export MAX_MCP_OUTPUT_TOKENS=50000
claude
```

This is useful for servers that:

- query large datasets
- return database schemas

- generate reports
- process extensive logs

The environment variable applies to tools that do not declare their own limit.

Tools that set:

```
anthropic/maxResultSizeChars
```

use that value for text content instead.

Image data remains subject to:

```
MAX_MCP_OUTPUT_TOKENS
```

Raise the Limit for a Specific Tool

MCP server authors can set:

```
_meta["anthropic/maxResultSizeChars"]
```

in the tool's `tools/list` response.

Example:

```
{
  "name": "get_schema",
  "description": "Returns the full database schema",
  "_meta": {
    "anthropic/maxResultSizeChars": 200000
  }
}
```

Claude Code raises the tool's persistence threshold to the requested size, up to:

```
500,000 characters
```

This is useful for tools that inherently return large results, such as:

- full database schemas
- large file trees

Without the annotation, oversized output may be persisted to disk and replaced in the conversation with a file reference.

The annotation applies independently of:

```
MAX_MCP_OUTPUT_TOKENS
```

for text content.

Image data is still subject to the token limit.

Tool Input Schemas with Root-Level Combinators

Some MCP servers define a tool input schema using a root-level JSON Schema combinator:

```
anyOf
oneOf
allOf
```

The Claude API does not accept these keywords at the schema root.

It does accept combinators nested inside:

```
properties
```

Claude Code can flatten root-level combinators into a single object before sending the schema to the API.

`allOf`

- properties from every branch are merged
- each branch's `required` list continues to apply

`anyOf` and `oneOf`

- properties from every branch are merged

- each branch's required combinations are described in the tool description rather than enforced directly by the flattened schema

The MCP server still receives the arguments Claude selected.

Therefore, the server should continue validating argument combinations itself.

If Claude Code cannot produce an API-compatible schema, it skips the affected tool, records the reason in the server log, and leaves the server's other tools available.

Require Approval for a Specific Tool

An MCP server can mark a tool as requiring explicit user approval on every call.

Set:

```
_meta["anthropic/requiresUserInteraction"]
```

to the JSON boolean:

```
true
```

Example:

```
{
  "name": "grant_access",
  "description": "Requests access to a protected resource",
  "_meta": {
    "anthropic/requiresUserInteraction": true
  }
}
```

Claude Code then shows a permission prompt every time the tool is called.

This applies even in permission modes such as:

```
acceptEdits
auto
bypassPermissions
```

Allow rules do not bypass the prompt.

In:

```
dontAsk
```

mode, where prompts are forbidden, the tool call is denied.

Use this for operations where explicit human approval is itself a requirement, such as:

- consent
- access grants
- protected authorization actions

Other tools from the same server keep their normal permission behavior.

In non-interactive mode with:

```
--permission-prompt-tool
```

an `allow` response for such a tool is converted into a denial because the required approval must reach a person.

The Agent SDK's:

```
canUseTool
```

callback can receive and approve these calls because the SDK host is expected to present the request to a user.

When connected through Remote Control or an SDK host, Claude Code marks the permission request as requiring user interaction so the client presents the full approval prompt.

Respond to MCP Elicitation Requests

MCP servers can request structured input from you while a task is running.

Claude Code automatically displays an interactive dialog and passes your response back to the server.

No additional configuration is required.

Servers can request input in two ways.

Form Mode

Claude Code displays a form with fields defined by the server.

Examples:

- username
- password
- configuration values

URL Mode

Claude Code opens a browser URL for:

- authentication
- approval
- authorization

Complete the flow and confirm in the CLI.

To automatically respond to elicitation requests without displaying a dialog, use the:

```
Elicitation
```

hook.

See the MCP elicitation specification for protocol details and schema examples.

Use MCP Resources

MCP servers can expose resources that you reference with `@` mentions, similar to files.

List Available Resources

Type:

```
@
```

in your prompt.

Resources from connected MCP servers appear alongside files in autocomplete.

Reference a Resource

Format:

```
@server:protocol://resource/path
```

Examples:

```
Can you analyze @github:issue://123 and suggest a fix?
```

```
Please review the API documentation at @docs:file://api/authentication
```

Reference Multiple Resources

```
Compare @postgres:schema://users with @docs:file://database/user-model
```

Notes

- Resources are automatically fetched and attached when referenced.
- Resource paths support fuzzy search in `@` autocomplete.
- Claude Code automatically provides tools for listing and reading MCP resources when the server supports them.
- Resources can contain text, JSON, structured data, and other server-provided content.

Scale with MCP Tool Search

Tool search reduces MCP context usage by deferring tool definitions until Claude needs them.

Only:

- tool names
- server instructions

need to load initially.

This means adding more MCP servers has relatively little impact on the context window.

Claude Code does not impose a fixed per-server tool cap.

The practical limit is the available context-window budget.

How Tool Search Works

Tool search is enabled by default.

MCP tools are deferred rather than loading their full schemas into context at startup.

When Claude needs a tool, it searches for the relevant MCP capability.

Only the tools Claude actually uses enter the context.

From the user's perspective, MCP tools work normally.

For threshold-based loading, set:

```
ENABLE_TOOL_SEARCH=auto
```

This loads schemas upfront when they fit within 10% of the context window and defers the rest.

Guidance for MCP Server Authors

With tool search enabled, the MCP server's instruction field becomes more important.

Server instructions should explain:

- what categories of tasks the tools handle
- when Claude should search for them
- the server's key capabilities

Claude Code truncates:

- tool descriptions
- server instructions

at approximately:

```
2 KB each
```

Keep them concise and put critical information near the beginning.

Configure Tool Search

Use:

```
ENABLE_TOOL_SEARCH
```

Values

Value	Behavior
Unset	All MCP tools deferred and loaded on demand
`true`	Force all MCP tools to be deferred
`auto`	Load upfront if tools fit within 10% of context
`auto:N`	Use a custom threshold percentage
`false`	Load all MCP tools upfront

Examples:

```
# Use a custom 5% threshold
ENABLE_TOOL_SEARCH=auto:5 claude

# Disable tool search
ENABLE_TOOL_SEARCH=false claude
```

You can also configure it in the `env` field of `settings.json`.

Tool search requires a model that supports:

```
tool_reference
```

blocks.

Haiku models do not support this capability.

Tool search may be disabled by default on:

- Google Cloud's Agent Platform
- environments where `ANTHROPIC_BASE_URL` points to a non-first-party host

This is because some proxies do not forward `tool_reference` blocks.

You can explicitly set:

```
ENABLE_TOOL_SEARCH
```

to override the default behavior.

Disable the `ToolSearch` Tool

```
{
  "permissions": {
    "deny": ["ToolSearch"]
  }
}
```

Exempt a Server from Tool Deferral

Set:

```
"alwaysLoad": true
```

to load all tools from a specific server into context at startup.

Example:

```
{
  "mcpServers": {
    "core-tools": {
      "type": "http",
      "url": "https://mcp.example.com/mcp",
      "alwaysLoad": true
    }
  }
}
```

Use this sparingly.

Every upfront tool consumes context that would otherwise remain available for the conversation.

An MCP server can also mark individual tools as always loaded:

```
{
  "_meta": {
    "anthropic/alwaysLoad": true
  }
}
```

Setting:

```
alwaysLoad: true
```

also causes startup to wait for the server to connect, up to the normal connection timeout.

This happens because the tool schemas must be available when the first prompt is built.

Other MCP servers can continue connecting in the background.

Use MCP Prompts as Commands

MCP servers can expose prompts that become Claude Code commands.

Discover Prompts

Type:

```
/
```

MCP prompts appear in the format:

```
/mcp__servername__promptname
```

Execute a Prompt Without Arguments

```
/mcp__github__list_prs
```

Execute a Prompt with Arguments

```
/mcp__github__pr_review 456
```

```
/mcp__jira__create_issue "Bug in login flow" high
```

Notes

- MCP prompts are dynamically discovered from connected servers.
- Arguments are parsed according to the prompt's parameter definitions.
- Prompt results are injected directly into the conversation.
- Server and prompt names are normalized, with spaces converted to underscores.

Managed MCP Configuration

Organizations can centrally control which MCP servers users may connect to.

Managed MCP configuration supports:

- deploying a fixed server set with `managed-mcp.json`
- restricting servers through `allowedMcpServers`
- blocking servers through `deniedMcpServers`
- controlling what users see when a server is blocked

See:

Managed MCP configuration